Procedure 1: Using Rasa

1. Install Python and Rasa, and create a new Rasa project.
2. Choose a conversational domain, such as a College Enquiry Chatbot.
3. Define intents such as greet, ask_courses, ask_fees, and ask_admission in the data/nlu.yml file.
4. Add multiple training phrases for each intent in data/nlu.yml.
5. Define at least one entity, such as course, to extract course names from user messages.
6. Define the chatbot's intents, entities, and responses in the domain.yml file.
7. Configure the chatbot's response rules in the data/rules.yml file.
8. Train the chatbot by executing the following command:

rasa train

9. Run the chatbot using the Rasa shell:

rasa shell

10. Enter sample messages and observe the detected intents and chatbot responses.
11. Write a Python program using the hashlib library to calculate the SHA-256 hash of a user message.
12. Store the generated hash in a log file instead of storing the original plain-text message.
13. Hash the same message twice to verify determinism and modify the message slightly to demonstrate the avalanche effect.
14. Integrate hashing into the chatbot pipeline, if required, so incoming messages are hashed before logging while the original text is used for intent recognition.
15. Test the chatbot using multiple utterances and calculate intent recognition accuracy by comparing predicted intents with expected intents.
16. Record the observations and summarize the chatbot's working and the role of hashing in protecting message logs.

Procedure 2: Using Dialogflow

1. Open the Dialogflow Console and sign in with a Google account.

2. Create a new agent and select the appropriate language and time zone.
3. Choose a conversational domain, such as a College Enquiry Chatbot.
4. Create intents such as greet, ask_courses, ask_fees, and ask_admission.
5. Add multiple training phrases to each intent so the chatbot can recognize different ways of asking questions.
6. Define at least one entity, such as course, to extract course names from user messages.
7. Configure suitable text responses for each intent.
8. Train the chatbot by saving the intents and allowing Dialogflow to process the training phrases automatically.
9. Test the chatbot using the built-in console by entering sample messages.
10. Record the detected intents and corresponding chatbot responses.
11. Write a Python program using the hashlib library to calculate the SHA-256 hash of a user message.
12. Store the generated hash in a log file instead of storing the original plain-text message.
13. Hash the same message twice to verify determinism and modify the message slightly to demonstrate the avalanche effect.
14. Optionally, integrate the hashing process with the chatbot application so that incoming messages are hashed before logging while the original text is used for intent recognition.
15. Test the chatbot with multiple utterances, compare the detected intents with the expected intents, and calculate intent recognition accuracy.
16. Record the observations and summarize the chatbot's working and the role of hashing in protecting message logs.

give me common points from both

Common Procedure for Rasa and Dialogflow

1. Select a simple conversational domain, such as a College Enquiry Chatbot.
2. Create a chatbot and define intents such as greeting, course enquiry, fees enquiry, and admission enquiry.
3. Add multiple training phrases for each intent.

4. Define at least one entity, such as course, to extract information from user messages.
5. Configure suitable responses for each intent.
6. Train the chatbot using the selected platform.
7. Test the chatbot by entering sample messages and record the detected intents and corresponding responses.
8. Write a Python program using the hashlib library to calculate the SHA-256 hash of a user input string.
9. Store the generated hash in a log file instead of storing the original plain-text message.
10. Hash the same input twice and verify that both hashes are identical, demonstrating determinism.
11. Modify the input slightly and calculate its hash again to demonstrate the avalanche effect.
12. Optionally, integrate hashing into the chatbot pipeline so that incoming messages are hashed before logging while the original messages are used for intent recognition.
13. Test the chatbot with multiple utterances and calculate intent recognition accuracy using the formula:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100$$

14. Record the observations and summarize the chatbot's working and the role of SHA-256 hashing in protecting message logs.

ORAL

12. Viva Questions and Answers

24. What is a chatbot, and how does it understand user messages?
A chatbot is a software application that communicates with users. It understands messages using Natural Language Understanding (NLU) and identifies the user's intent.

25. What is the difference between Dialogflow and Rasa?
Dialogflow is a Google platform that provides a graphical interface for building chatbots, whereas Rasa is an open-source framework used to develop customizable chatbots.

26. What is an intent, and what are training phrases used for?
An intent represents the user's purpose. Training phrases are example messages used to teach the chatbot to recognize different intents.

27. What is an entity in the context of a chatbot?
An entity is specific information extracted from a user message, such as a course name, date, or location.

28. What is the difference between NLU and dialogue management?
NLU understands the user's message and identifies its intent and entities. Dialogue management decides how the chatbot should respond and what action it should take next.

29. What is hashing, and how is it different from encryption?
Hashing converts data into a fixed-length hash, while encryption converts data into a form that can be reversed using the appropriate key.

30. Why is hashing considered a one-way function?
Hashing is considered one-way because the original input cannot practically be reconstructed from its hash.

31. What is the avalanche effect in hashing?
The avalanche effect means that a small change in the input produces a substantially different hash.

32. Why should user inputs be hashed before logging or storage?
Hashing reduces direct exposure of the original messages in logs and helps protect user privacy. However, common messages may still be guessed.

33. What are the limitations of hashing as a security measure for a chatbot system?
Hashing does not encrypt data, guarantee complete privacy, or prevent unauthorized access to the chatbot. Common inputs may be guessed, and other security measures are still required.